RADIX HASH JOIN PROJECT

Radix partitioning: bits, passes and page size

Apple M5 · 10 logical CPUs, L1 128 KiB, L2 16 MiB

We split 16,777,216 records into 2^B partitions for B = 0 to 20, in 1, 2 or 3 passes. One pass costs about 1.7 ns per record while it writes to only a few partitions. The cost steps up at 1 bits (2 partitions) and stays above twice the flat level from 18 bits. Two passes only become cheaper than one from 18 bits. The paper's reason for multi-pass partitioning is still visible, but much later than a 64-entry TLB, typical in 1999, would suggest (6 bits).

1.7 ns	2	18 bits
per record, one pass, few partitions	partitions where one pass gets slower (1 bits)	where two passes start to beat one

1 What the paper says

Radix partitioning reads each record, takes some bits of its hash and copies the record to the partition those bits name. Each partition is a separate place in memory to write to. When there are more partitions than pages the TLB can translate quickly, or than cache lines the CPU can keep open for writing, almost every copy misses. Boncz, Manegold and Kersten therefore split B bits over P passes, so each pass only writes to $2^{B/P}$ places. Extra passes cost extra reads and writes, so the best number of passes depends on where the memory system stops coping with the fan-out.

2 Method

- **Input.** The left table of the simple hash join experiment: 16,777,216 records of 8 bytes (128 MiB), keys shuffled with seed 42.
- **Radix value.** The lowest B bits of the same 64-bit hash the simple hash join uses. With several passes, pass 1 splits on the top group of those bits and later passes split each cluster further. Bits per pass are as even as possible (14 bits in 2 passes is 7 + 7).
- **Timing.** Each pass counts records per partition (histogram), then copies them (scatter); both are timed. All memory is allocated and touched before timing. One thread; macOS cannot pin threads to CPUs; threads ran at the highest priority class, which prefers the performance cores.
- **Repetitions.** One verified warmup, then 15 recorded runs of every setting. Each round runs every setting once, in a newly shuffled order, so a slow period affects all settings alike. We report medians; chart bands span the middle half of the runs (interquartile range). Typical run-to-run variation was 4% of the mean (coefficient of variation, median over settings).
- **Page size.** The system default (16 KiB); this OS does not let a program choose huge pages, so the page-size comparison was skipped.
- **Interference.** Hypervisor steal time was not recorded.
- **Correctness.** After the warmup every result is checked: each record is in the partition its hash names, the partitions cover all rows, and the records are unchanged. The --test mode also compares sorted output with the input for 238 small cases.

3 One pass: cost as the number of partitions grows

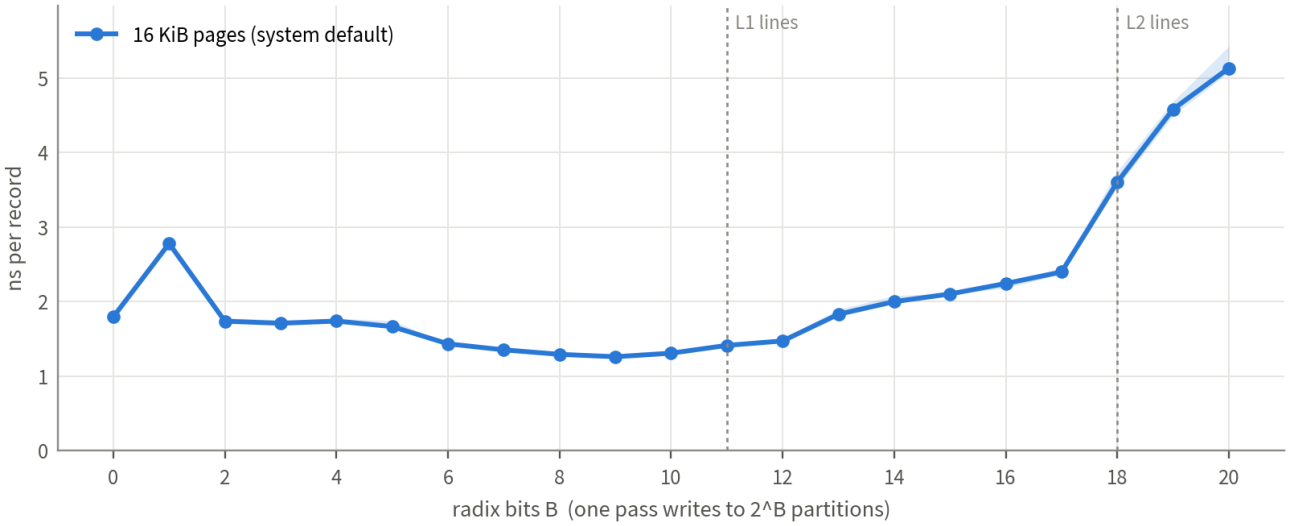


Figure 1. Median cost per record of a single pass; the band spans the middle half of the recorded runs. Dashed lines: where 2^B output cache lines (64 B each) would fill L1 and L2.

Up to 0 bits a pass costs about as much as a plain copy (1.80 ns per record at B = 0). At 1 bits (2 partitions) the cost steps up to 2.78 ns, rises slowly to 2.24 ns at 16 bits, and then climbs steeply to 5.14 ns at 20 bits.

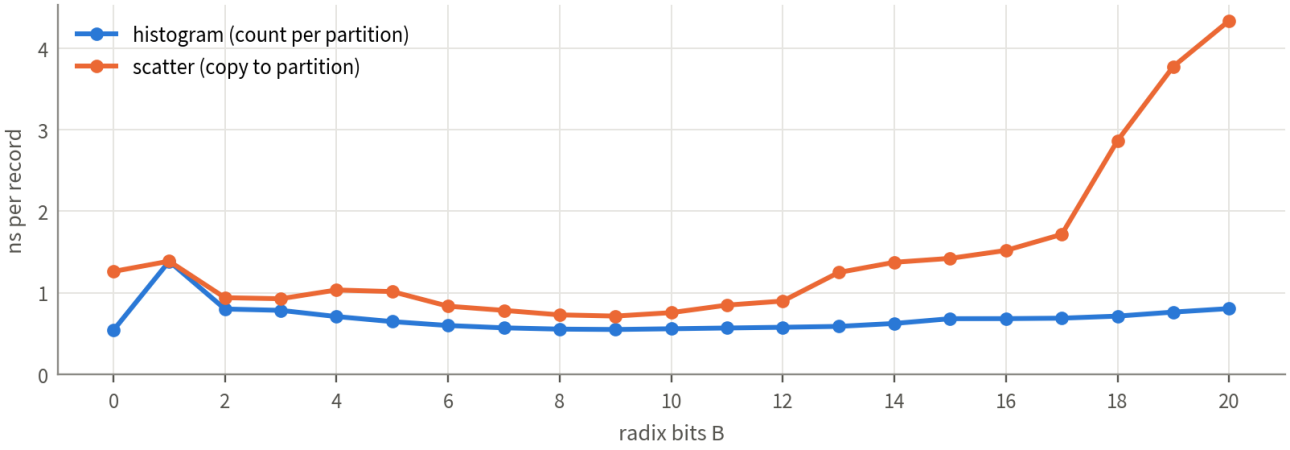


Figure 2. Histogram and scatter time of a single pass (medians, 4 KiB pages). The histogram only reads the input and updates a small array of counters; the scatter is where the cost grows.

4 More passes

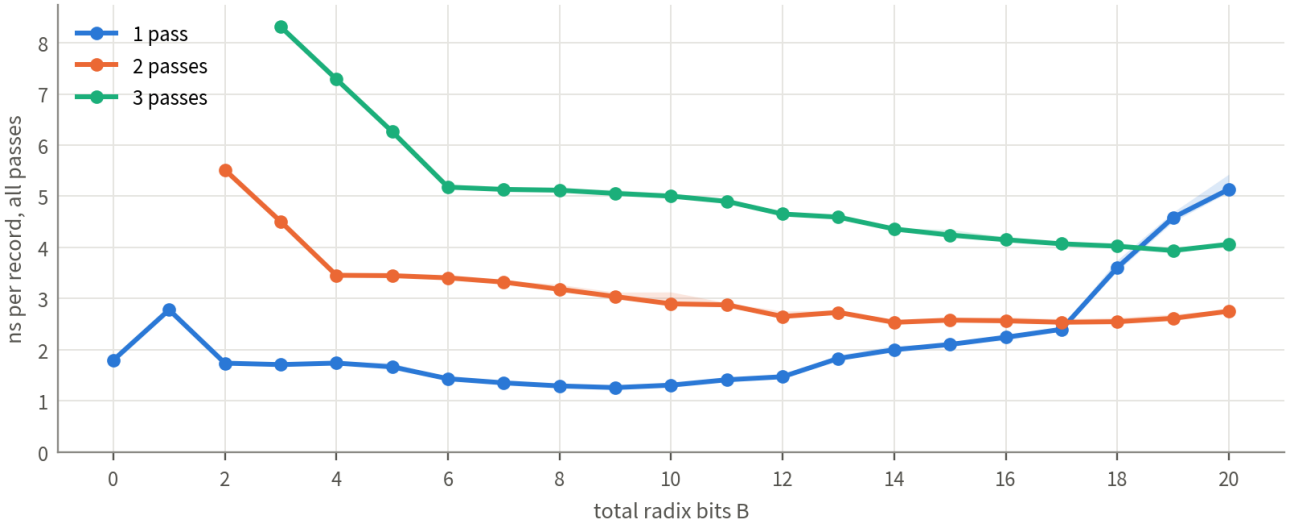


Figure 3. Median cost per record of all passes together; bands span the middle half of the recorded runs.

A second pass reads and writes all the data again, so it only pays once one large pass costs more than two small ones. That happens from 18 bits: at 20 bits one pass costs 5.14 ns and two passes 2.75 ns. Three passes were never the fastest choice up to 20 bits.

Bits B	Partitions	1 pass	2 passes	3 passes	Fastest
0	1	1.80	-	-	1 pass
1	2	2.78	-	-	1 pass
2	4	1.74	5.52	-	1 pass
3	8	1.71	4.51	8.31	1 pass
4	16	1.74	3.46	7.29	1 pass
5	32	1.67	3.45	6.26	1 pass
6	64	1.43	3.41	5.18	1 pass
7	128	1.35	3.32	5.13	1 pass
8	256	1.29	3.18	5.12	1 pass
9	512	1.26	3.04	5.05	1 pass
10	1,024	1.31	2.90	5.00	1 pass
11	2,048	1.41	2.88	4.90	1 pass
12	4,096	1.47	2.65	4.65	1 pass
13	8,192	1.83	2.73	4.59	1 pass
14	16,384	2.00	2.54	4.36	1 pass
15	32,768	2.10	2.58	4.24	1 pass
16	65,536	2.24	2.57	4.15	1 pass (tie)
17	131,072	2.40	2.54	4.07	1 pass
18	262,144	3.61	2.55	4.02	2 passes (tie)
19	524,288	4.58	2.61	3.94	2 passes
20	1,048,576	5.14	2.75	4.06	2 passes

Median ns per record over 15 runs, 16 KiB pages (system default). 'Tie': the runs of another pass count overlap with the fastest one, so the difference is within run-to-run noise. Shaded rows: more than one pass was fastest. Full statistics are in summary.csv.

5 Checking the cost model idea

The paper models each pass separately: its cost depends on how many partitions that pass writes to and how that compares with the cache and TLB sizes. If that is right, P passes should cost the sum of the single-pass costs at each pass's own bit count. Figure 4 makes that prediction from the measured one-pass curve, without fitting anything.

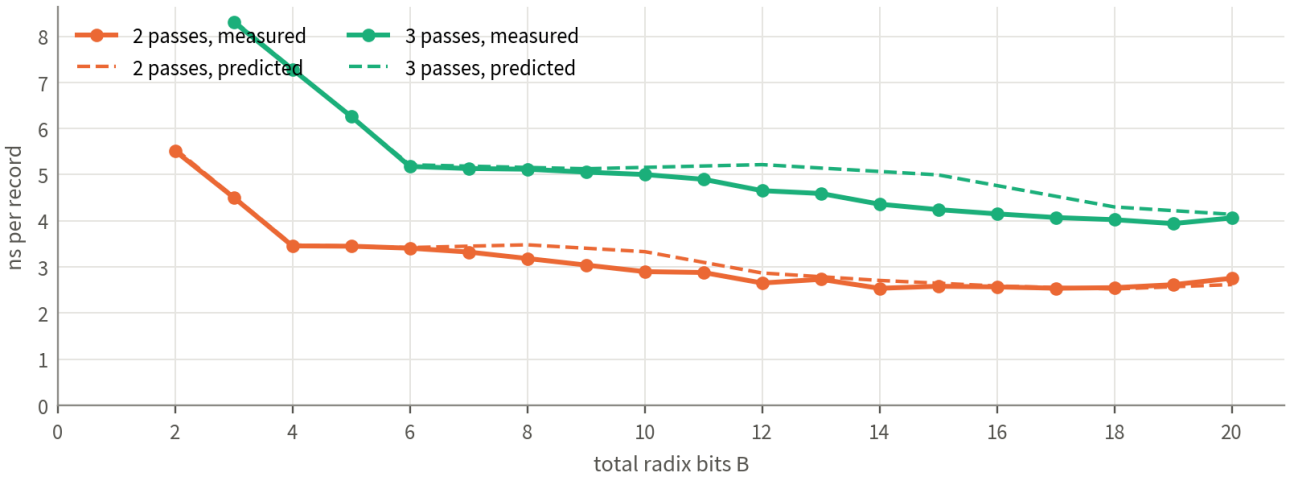


Figure 4. Measured multi-pass cost (solid) against the sum of single-pass costs (dashed).

Average prediction error: 2 passes 4% (largest 15%); 3 passes 6% (largest 18%). Later passes work on clusters that are already small, and a cluster that fits in cache is cheap to read again, so the sum tends to overestimate slightly.

6 What this means for a radix hash join

- One pass is close to copy speed up to 0 bits and stays under 3 ns per record up to 14 bits, enough to cut tens of millions of records into partitions of a few thousand.
- Two passes only help beyond about 18 bits, so a radix hash join on this machine should mostly use one pass, and two only for very large partition counts.
- The paper's main idea, that one pass must not write to too many places, still holds, but the limit is much higher than on 1999 hardware.

7 Limitations

- **Hardware counters** were not available on this machine. Without them we can see where the cost rises, not which cache or TLB causes it.
- **15 runs per setting.** Differences smaller than the run-to-run variation (about 4%) should be read as ties.
- **Uniform keys.** Partitions are almost equal in size. Skewed keys would make some much larger.

8 Future work

- **Miss counts.** Measure L1, L2 and TLB misses (perf on native Linux, Instruments on macOS) to confirm which limit causes the cost step.
- **Page size.** Isolate the TLB effect with huge pages on native Linux, where the page size can be chosen.
- **Write-combining.** Collect each partition's records in a cache-line buffer before writing them out; this modern technique attacks the same fan-out problem as extra passes.
- **Full cost model.** Measure per-level memory latencies and predict pass costs from cache and TLB sizes instead of from the measured one-pass curve.
- **Skewed keys.** Repeat with Zipf-distributed keys, which make partitions uneven.
- See docs/future-work.md for details.

9 Rerun

```
make
./bin/radix_partition --test
./bin/radix_partition results/radix-partitioning-new
python3 analysis/analyze_partition.py results/radix-partitioning-new
```

[1] P. Boncz, S. Manegold and M. L. Kersten. Database Architecture Optimized for the New Bottleneck: Memory Access. Proceedings of VLDB, 1999. Sections 3 and 4.